Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №2
з дисципліни «Методи та технології паралельного програмування»

Тема: «Механізми взаємодії та синхронізації потоків»

Варіант №4

Виконав:
студент 1 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
асистент, Лесик Валентин Олександрович

Дата здачі: 19.03.2026

КИЇВ – 2026

Мета роботи. Ознайомитися з механізмами взаємодії та синхронізації потоків, набути практичних навичок реалізації паралельних алгоритмів з координацією роботи між потоками.

Завдання. Реалізувати послідовну та паралельну мультипоточну обробку незалежних задач, за допомогою використання відповідних патернів паралельного програмування. Необхідно обрати відповідний фреймворк, описати його можливості та обмеження, пояснити причину вибору фреймворку відповідно описаним обмеженням. Мова програмування обирається на розгляд студента. Результати роботи повинні містити виконання логіки програми без застосування паралелізації та із застосуванням паралелізації за різних гіперпараметрів: кількості потоків, процесів, worker-ів; для кожного рішення треба знайти час виконання та зробити висновки, що впливає на прискорення та які треба підбирати гіперпараметри. У відповідях на контрольні запитання слід вказувати на особливості та відмінності обраного студентом фреймворку, так як в залежності від обраної мови буде різнитися реалізація, підходи та можливості застосування.

Задача 1: ***Порівняння патернів Map-Reduce, Fork-Join, Worker Pool.***

Реалізувати наступні задачі із використанням кожного патерну:

1. Підрахувати частоту появи тегів у наборі `HTML` документів (аналог підрахунок тегів по сайтах, але без вебскрапінгу). Згенерувати або завантажити набір даних із `HTML` документами, як мінімум 1000 (більше – краще) документів різного розміру та наповнення (можна допускати копії файлів), реалізувати методи без паралелізації та із застосуванням патернів;
2. Знайти мінімум, максимум, медіану та середнє значення масиву із $>$ 1000000 чисел. Згенерувати масив будь-яким випадковим чином (можна використати розподіли, що відмінні від нормального);
3. Помножити дві матриці великого розміру із $>$ 1000 чисел.

Для патернів можна використовувати готові рішення із обраних фреймворків. Проаналізувати переваги та недоліки кожного патерну на задачах, проаналізувати швидкодію, вибрати патерн, що підходить до задач найбільше, зробити висновки. Реалізація чи використання патерну не є принциповим, якщо є відповідний опис та аргументація його конкретного застосування у висновках.

Задача 2: ***Pipeline, Producer-Consumer.***

Реалізувати ОДНУ З наступних задач із використанням патернів:

- Система обробки фінансових транзакцій (або замовлень).
- Відео-рендеринг *або* Обробка зображень (Фільтри).

До першої задачі: згенерувати (можна із абсолютним сміттям, лише притримуватись типу даних) або завантажити великий файл, або тисячі невеликих файлів із даними по транзакціям. Транзакції повинні бути структуровані як «`ID_користувача, сума, валюта, дата, тип_товару`» (або запропонувати свою структуру). Потрібно реалізувати обидва патерни, що виконують задачі: конвертація валюти, повернення частини коштів за статусом користувача (проста реалізація – > ID, то 20% повернення коштів), агрегація – підрахувати фінальну суму. Реалізація чи використання патерну не є принциповим, якщо є відповідний опис та аргументація його конкретного застосування у висновках.

До другої задачі: згенерувати або завантажити набір кадрів (JPEG, PNG, RAW файли). Для кожного кадру треба: декодувати, накласти фільтр (складний як напр. Blur, або простий, як чорно-біле представлення); накласти водяний знак, закодувати назад. Потрібно реалізувати обидва патерни, що виконують ці задачі. Реалізація чи використання патерну не є принциповим, якщо є відповідний опис та аргументація його конкретного застосування у висновках.

Виміряти та проаналізувати найкращий підхід відносно часу виконання та додаткових переваг патерну.

Вимоги до звіту. Повинна бути титульна сторінка, тема, мета, завдання, короткий опис обраного підходу до рішення задачі, опис критичних моментів рішення, код проекту (лістинг), обґрунтування обрання фреймворку, опис відмінностей та переваг, скріншоти із демонстрацією виконання програми, скріншоти із демонстрацією прискорення та різними значеннями параметрів, відповіді на контрольні запитання, висновки.

Хід роботи.

Задача 1: Підрахунок HTML-тегів. Опис підходу до вирішення.

Задача підрахунку частоти появи тегів у великому масиві HTML-документів є класичним прикладом проблеми, що ідеально піддається розпаралелюванню на рівні даних. Оскільки кожен документ може бути оброблений абсолютно незалежно від інших, це дозволяє уникнути складних механізмів синхронізації під час фази аналізу. Для вирішення цієї задачі було застосовано три архітектурні патерни: Map-Reduce, Fork-Join та Worker Pool.

Патерн Map-Reduce реалізовано через трансформацію кожного документа у локальний словник частот (фаза Map) з подальшим злиттям усіх словників у єдиний фінальний результат (фаза Reduce). Патерн Fork-Join імплементовано за до-

помогою рекурсивного поділу масиву документів навпіл. Коли розмір підмасиву досягає заданого порогу (threshold), документи обробляються послідовно, після чого результати рекурсивно об'єднуються (Join). Патерн Worker Pool використано для розподілу пулу паралельних ітераторів між фіксованою кількістю робочих потоків, що дозволяє жорстко контролювати споживання ресурсів процесора.

Критичні моменти рішення.

Найбільшим архітектурним викликом у цій задачі є об'єднання результатів. Наївний підхід полягав би у створенні єдиного глобального словника (HashMap), захищеного м'ютексом, куди кожен потік записував би знайдені теги. Однак такий підхід миттєво створив би «пляшкове горло»: потоки постійно блокували б один одного, очікуючи на зняття блокування, що звело б нанівець усі переваги паралелізму і зробило б програму повільнішою за послідовну.

Для уникнення цієї проблеми всі три патерни реалізовано за принципом ізольованого стану. Кожна паралельна задача створює, наповнює та повертає свій власний незалежний словник. Злиття цих словників відбувається попарно та ієрархічно на етапах об'єднання (Reduce або Join). Хоча це вимагає додаткових витрат пам'яті на створення багатьох хешмапів та часу на їхнє злиття, відсутність блокувань робить загальний час виконання в рази меншим. Крім того, у патерні Fork-Join критичним є правильний вибір порогу рекурсії, щоб не витрачати ресурси системи на розгалуження для занадто малих масивів даних.

Аналіз результатів та підбір гіперпараметрів.

Згідно з отриманими логами (див. скріншот нижче), послідовне виконання задачі на одному ядрі зайняло 0.3668 секунд. Застосування всіх трьох паралельних патернів дало значне прискорення, проте їхня ефективність дещо відрізнялася через специфіку роботи планувальника та накладні витрати.

```
=== REPORT FOR HTML TAG COUNTING ===
Sequential: 0.3668 seconds.
Map-Reduce with 8 partitions: 0.0588 seconds.
Fork-Join with threshold of 100: 0.0602 seconds.
Fork-Join with threshold of 10: 0.0611 seconds.
Worker Pool with 2 workers: 0.2033 seconds.
Worker Pool with 4 workers: 0.1296 seconds.
Worker Pool with 8 workers: 0.0735 seconds.
```

Абсолютним лідером став патерн Map-Reduce з результатом 0.0588 секунди, що майже в 6 разів швидше за послідовний код. Патерн Fork-Join показав практично ідентичну швидкодію: 0.0602 секунди при порозі відсікання у 100 документів та 0.0611 секунди при порозі у 10 документів. Близькість цих результатів свідчить про те, що бібліотека Rayon під капотом зводить Map-Reduce до того ж самого алгоритму Fork-Join, а надзвичайно низькі накладні витрати планувальника дозволяють ефективно працювати навіть із дуже дрібними порціями даних (поріг 10), не втрачаючи процесорного часу на постійне створення та розподіл нових підзадач. Патерн Worker Pool продемонстрував очікувану динаміку класичного масштабування: 0.2033 секунди для 2 воркерів, 0.1296 секунди для 4 та 0.0735 секунд для 8 робочих потоків. Це доводить, що для досягнення максимальної продуктивності у подібних задачах кількість потоків у пулі має чітко дорівнювати або максимально наближатися до кількості фізичних ядер процесора, інакше система просто не розкриває свій повний паралельний потенціал.

```
[2026-03-19 01:24 INFO] TASK: HTML Tag Counting, MODE: Sequential, UNIQUE TAGS PROCESSED: 9
[2026-03-19 01:24 INFO] TASK: HTML Tag Counting, MODE: Map-Reduce with 8 partitions, UNIQUE TAGS PROCESSED: 9
[2026-03-19 01:24 INFO] TASK: HTML Tag Counting, MODE: Fork-Join with threshold of 100, UNIQUE TAGS PROCESSED: 9
[2026-03-19 01:24 INFO] TASK: HTML Tag Counting, MODE: Fork-Join with threshold of 10, UNIQUE TAGS PROCESSED: 9
[2026-03-19 01:24 INFO] TASK: HTML Tag Counting, MODE: Worker Pool with 2 workers, UNIQUE TAGS PROCESSED: 9
[2026-03-19 01:24 INFO] TASK: HTML Tag Counting, MODE: Worker Pool with 4 workers, UNIQUE TAGS PROCESSED: 9
[2026-03-19 01:24 INFO] TASK: HTML Tag Counting, MODE: Worker Pool with 8 workers, UNIQUE TAGS PROCESSED: 9
```

Задача 2: Обчислення статистичних метрик великого масиву. Опис підходу до вирішення.

Ця задача вимагає обчислення чотирьох статистичних метрик (мінімуму, максимуму, середнього значення та медіани) для масиву, що містить понад мільйон чисел з плаваючою комою. Для задоволення вимоги щодо використання ненормального розподілу, масив було згенеровано на основі рівномірного розподілу (Uniform distribution) за допомогою генератора псевдовипадкових чисел. Архітектурно задача поділяється на дві складові. Знаходження мінімуму, максимуму та суми елементів (для середнього значення) потребує лише одного проходу по масиву. Ця частина ідеально лягає на патерни агрегації Map-Reduce та Fork-Join. Натомість обчислення медіани вимагає попереднього сортування масиву, що є значно важчою алгоритмічною операцією. Для сортування було використано вбудований у фреймворк Rayon алгоритм паралельного сортування, який під капотом реалізує патерн Fork-Join.

Критичні моменти рішення.

Головним критичним моментом стало уникнення багаторазових проходів

по величезному масиву даних. Щоб не виконувати окремий прохід для мінімуму, окремий для максимуму і окремий для суми, було створено комплексну структуру-акумулятор. Ця структура збирає всі три метрики одночасно за один прохід, а її функція злиття (Reduce) коректно об'єднує локальні екстремуми та суми від різних потоків в єдиний результат, унеможливлюючи стани гонитви (data races) без використання м'ютексів.

Другим викликом стала робота з медіаною. Оскільки сортування змінює порядок елементів, а бенчмарки виконуються послідовно один за одним на одному і тому ж об'єкті, сортування оригінального масиву призвело б до спотворення вихідних даних для наступних тестів. Тому перед обчисленням медіани алгоритм виконує клонування всього масиву. Це створює додаткове навантаження на шину оперативної пам'яті, проте гарантує чистоту експерименту та повну ізоляцію стану між різними архітектурними патернами.

Аналіз результатів та підбір гіперпараметрів.

Як видно на скріншотах нижче, всі реалізовані патерни коректно впоралися із завданням, успішно обчисливши статистичні показники для масиву. Аналіз часу виконання демонструє очікувану картину для комбінованої задачі, де домінує час виділення пам'яті та сортування.

```
=== REPORT FOR MIN, MAX, MEAN, AVERAGE OF THE ARRAY ===
Sequential: 0.2956 seconds.
Map-Reduce with 8 partitions: 0.0890 seconds.
Fork-Join with threshold of 10000: 0.0955 seconds.
Fork-Join with threshold of 1000: 0.0880 seconds.
Worker Pool with 2 workers: 0.2007 seconds.
Worker Pool with 4 workers: 0.1439 seconds.
Worker Pool with 8 workers: 0.1060 seconds.
```

Послідовне виконання на одному ядрі зайняло 0.2956 секунд. Застосування паралелізму дозволило прискорити роботу більш ніж утричі. Патерни Map-Reduce та Fork-Join показали майже однакову максимальну швидкодію: 0.0890 секунд та 0.0880 секунд відповідно. Важливо відзначити, що зміна порогу відсікання у Fork-Join з 10000 до 1000 елементів практично не вплинула на загальний результат (0.0955 секунд). Це підтверджує, що планувальник задач має надзвичайно малі накладні витрати на управління, і алгоритм не страждає від надмірного рекурсивного розгалуження.

Патерн Worker Pool продемонстрував чітку динаміку масштабування. Збільшення кількості робочих потоків з 2 до 4 скоротило час із 0.2007 до 0.1439 секунди, а перехід до 8 воркерів дав результат 0.1060 секунди, що максимально наблизилося до показників інших патернів. Оптимальним гіперпараметром для цієї задачі є використання пулу потоків, що дорівнює кількості фізичних ядер процесора, оскільки операції швидкого сортування вимагають інтенсивних математичних обчислень та постійного переміщення даних у процесорних кешах.

```
2026-19-03.log ×
MODE: Sequential, STATS: Min: -1000.00, Max: 1000.00, Mean: 0.15, Median: 0.29
MODE: Map-Reduce with 8 partitions, STATS: Min: -1000.00, Max: 1000.00, Mean: -0.38, Median: -0.12
MODE: Fork-Join with threshold of 10000, STATS: Min: -1000.00, Max: 1000.00, Mean: 0.11, Median: 0.11
MODE: Fork-Join with threshold of 1000, STATS: Min: -1000.00, Max: 1000.00, Mean: -0.07, Median: -0.02
MODE: Worker Pool with 2 workers, STATS: Min: -1000.00, Max: 1000.00, Mean: 0.08, Median: 0.07
MODE: Worker Pool with 4 workers, STATS: Min: -1000.00, Max: 1000.00, Mean: -0.16, Median: -0.30
MODE: Worker Pool with 8 workers, STATS: Min: -1000.00, Max: 1000.00, Mean: -0.25, Median: -0.62
```

Задача 3: Множення матриць великого розміру. Опис підходу до вирішення.

Задача множення двох квадратних матриць розміром 1000×1000 елементів вимагає виконання одного мільярда математичних операцій, оскільки її алгоритмічна складність становить $O(N^3)$. Це робить її надзвичайно інтенсивною як для процесора (обчислення), так і для підсистеми пам'яті (читання).

Головним підходом до оптимізації стало попереднє транспонування другої матриці перед початком множення. Замість класичного проходу по стовпцях, який спричиняє масові промахи кешу (cache misses) через непослідовне читання пам'яті, транспонування дозволяє читати обидві матриці суворо по рядках. Після цього застосовано патерни просторового розпаралелювання. У патерні Map-Reduce кожен рядок першої матриці незалежно множиться на всі рядки транспонованої другої матриці (фаза Map), а отримані результуючі рядки зливаються в єдиний одновимірний масив (фаза Reduce). Патерн Fork-Join реалізовано шляхом рекурсивного поділу діапазону індексів рядків першої матриці навпіл, поки розмір блоку не досягне заданого порогу.

Критичні моменти рішення.

Найбільшим архітектурним викликом під час застосування чистих патернів Map-Reduce та Fork-Join до великих масивів є проблема перерозподілу пам'яті (memory reallocation). Оскільки ми свідомо відмовилися від використання єдиного масиву із захистом через м'ютекси для уникнення блокувань, кожен потік генерує власні незалежні вектори-рядки.

На етапі злиття результатів (Reduce або Join) алгоритм змушений використовувати метод розширення векторів, який під капотом може викликати системні виклики для виділення нових блоків оперативної пам'яті та копіювання даних. Хоча це гарантує абсолютну безпеку потоків (thread safety) та відсутність станів гонитви, операції копіювання масивів стають новим, хоч і значно меншим, «пляшковим горлом». Саме тому швидкість виконання сильно залежить від ефективності внутрішнього аллокатора пам'яті операційної системи.

Аналіз результатів та підбір гіперпараметрів.

Результати бенчмаркінгу демонструють високу ефективність обраної архітектури та відмінне масштабування. Послідовне виконання алгоритму на одному ядрі зайняло 0.5000 секунди. Застосування паралельних патернів дозволило скоротити цей час більш ніж у шість разів.

```
=== REPORT FOR MATRICES MULTIPLICATION ===
Sequential: 0.5000 seconds.
Map-Reduce with 8 partitions: 0.0816 seconds.
Fork-Join with threshold of 100: 0.1001 seconds.
Fork-Join with threshold of 10: 0.0794 seconds.
Worker Pool with 2 workers: 0.2621 seconds.
Worker Pool with 4 workers: 0.1765 seconds.
Worker Pool with 8 workers: 0.1063 seconds.
```

Найкращий результат показав патерн Fork-Join із порогом відсікання у 10 рядків – 0.0794 секунди. Цікаво, що зменшення порогу з 100 до 10 не сповільнило, а навпаки, трохи прискорило програму (з 0.1001 до 0.0794 секунди). Це свідчить про те, що фреймворк Rayon має екстремально оптимізований механізм створення легковагових задач, і дрібніше розбиття дозволило планувальнику краще збалансувати навантаження між ядрами (work-stealing) на фінальних етапах обчислень. Патерн Map-Reduce з 8 партиціями показав практично ідентичний найкращому час – 0.0816 секунди.

Патерн Worker Pool знову продемонстрував класичну лінійну залежність швидкості від кількості виділених апаратних ресурсів: 0.2621 секунди для 2 потоків, 0.1765 секунди для 4 потоків та 0.1063 секунди для 8 потоків. Оптимальними гіперпараметрами для подібних інтенсивних математичних обчислень є використання дрібного розміру чанку (порогу рекурсії) для забезпечення максимальної

гнучкості балансувальника, а також використання кількості потоків, що дорівнює кількості фізичних ядер процесора.

```
MODE: Sequential, FIRST ELEMENT VALUE: 1416.99
MODE: Map-Reduce with 8 partitions, FIRST ELEMENT VALUE: -729.35
MODE: Fork-Join with threshold of 100, FIRST ELEMENT VALUE: -250.20
MODE: Fork-Join with threshold of 10, FIRST ELEMENT VALUE: 1802.79
MODE: Worker Pool with 2 workers, FIRST ELEMENT VALUE: 105.05
MODE: Worker Pool with 4 workers, FIRST ELEMENT VALUE: -37.10
MODE: Worker Pool with 8 workers, FIRST ELEMENT VALUE: 262.29
```

Задача 4: Конвеєрна обробка відео (Pipeline та Producer-Consumer). Опис підходу до вирішення.

Для реалізації системи обробки потокових медіаданих було розроблено програму накладання графічних ефектів (чорно-білий фільтр та напівпрозорий водяний знак) на кадри реального відео. Процес обробки включає підготовку (розкадровку відеофайлу за допомогою FFmpeg), чотири логічні стадії обробки (декодування з диска, градації сірого, альфа-композитинг водяного знака, кодування назад на диск у форматі PNG) та фінальну збірку оброблених кадрів у готовий відеофайл формату MP4. Для розпаралелювання було застосовано два архітектурні патерни. Патерн Pipeline (Конвеєр) реалізовано через виділення окремого потоку під кожну стадію обробки. Ці потоки з'єднані обмеженими каналами (bounded channels) з бібліотеки crossbeam, що запобігає переповненню оперативної пам'яті (backpressure) у разі різної швидкості роботи стадій. Патерн Producer-Consumer працює через спільну чергу завдань, куди потік-виробник передає шляхи до кадрів. Потоки-споживачі (workers) конкурують за ці завдання, і кожен воркер самостійно виконує весь цикл обробки (усі чотири стадії) для отриманого кадру.

Критичні моменти рішення.

Важливим інженерним викликом стала інтеграція системних викликів FFmpeg та задоволення математичних обмежень відеокодеків. Зокрема, стандартний кодек H.264 вимагає строго парних значень ширини та висоти кадру. Оскільки автоматичне масштабування могло призвести до непарної висоти (наприклад, 427 пікселів), проблему було вирішено за допомогою апаратного відеофільтра pad під час фінальної збірки, який динамічно округлює розмір до парних значень. У багатопотоковій архітектурі критичним моментом виявилося управління життєвим циклом каналів. У патерні Producer-Consumer необхідно було явно закривати (drop) оригінальні кінці каналів (Sender) у головному потоці після завершення диспетчеризації завдань. Без цього кроку ітератори приймачів у робочих потоках

назавжди блокувалися б в очікуванні нових даних, що призводило до взаємного блокування (deadlock) усієї програми на етапі збору результатів.

Аналіз результатів та порівняння патернів.

Згідно з отриманими метриками бенчмаркінгу, послідовне виконання алгоритму для 288 кадрів зайняло 5.4461 секунди. Патерн Pipeline з використанням 4 потоків (по одному на кожну логічну стадію) скоротив цей час до 1.8711 секунди. Хоча прискорення є значним, загальна пропускна здатність конвеєра строго обмежується швидкістю виконання найповільнішої стадії (у даному випадку це стадії I/O та кодування). Через це швидші потоки періодично простоюють в очікуванні звільнення буфера, і система не досягає максимальної теоретичної продуктивності.

```
[libx264 @ 0x5582c1c83280] kb/s:969.76
=== REPORT FOR VIDEO PROCESSING PIPELINE ===
Sequential: 5.4461 seconds.
Pipeline with buffer capacity of 5 and max 4 threads: 1.8711 seconds.
Producer-Consumer with buffer capacity of 20, 2 consumers and 1 producers: 2.6792 seconds.
Producer-Consumer with buffer capacity of 20, 4 consumers and 1 producers: 1.4840 seconds.
Producer-Consumer with buffer capacity of 20, 8 consumers and 1 producers: 0.8310 seconds.
```

Патерн Producer-Consumer продемонстрував значно кращу динаміку масштабованості завдяки повністю незалежному розподілу загальної задачі. При використанні 2 потоків-споживачів час склав 2.6792 секунди. Перехід до 4 споживачів зменшив час до 1.4840 секунди, що дозволило цьому патерну успішно випередити показники конвеєра. При використанні 8 воркерів було досягнуто максимальної продуктивності у 0.8310 секунди, що еквівалентно більш ніж 6.5-кратному прискоренню порівняно з базовим послідовним кодом. Ці результати наочно доводять, що для ресурсоємних I/O та обчислювальних задач, де елементи (кадри) можна обробляти повністю незалежно один від одного, архітектура єдиного пулу потоків з балансуванням навантаження є значно гнучкішою та ефективнішою за жорстко сегментований конвеєр.

```
[2026-03-19 19:41 INFO] Extracting frames from video using FFmpeg...
[2026-03-19 19:41 INFO] FFmpeg exited with status: exit status: 0
[2026-03-19 19:41 INFO] TASK: Video Processing Pipeline, MODE: Sequential, PROCESSED FRAMES: 288
[2026-03-19 19:41 INFO] Compiling processed frames back into video...
[2026-03-19 19:41 INFO] Video successfully saved to: "processed_video.mp4"
[2026-03-19 19:41 INFO] Extracting frames from video using FFmpeg...
[2026-03-19 19:41 INFO] FFmpeg exited with status: exit status: 0
[2026-03-19 19:41 INFO] MODE: Pipeline with buffer capacity of 5 and max 4 threads, PROCESSED FRAMES: 288
[2026-03-19 19:41 INFO] Compiling processed frames back into video...
[2026-03-19 19:41 INFO] Video successfully saved to: "processed_video.mp4"
[2026-03-19 19:41 INFO] Extracting frames from video using FFmpeg...
[2026-03-19 19:42 INFO] FFmpeg exited with status: exit status: 0
[2026-03-19 19:42 INFO] MODE: Producer-Consumer with 2 consumers and 1 producers, PROCESSED FRAMES: 288
[2026-03-19 19:42 INFO] Compiling processed frames back into video...
[2026-03-19 19:42 INFO] Video successfully saved to: "processed_video.mp4"
[2026-03-19 19:42 INFO] Extracting frames from video using FFmpeg...
[2026-03-19 19:42 INFO] FFmpeg exited with status: exit status: 0
[2026-03-19 19:42 INFO] MODE: Producer-Consumer with 4 consumers and 1 producers, PROCESSED FRAMES: 288
[2026-03-19 19:42 INFO] Compiling processed frames back into video...
[2026-03-19 19:42 INFO] Video successfully saved to: "processed_video.mp4"
[2026-03-19 19:42 INFO] Extracting frames from video using FFmpeg...
[2026-03-19 19:42 INFO] FFmpeg exited with status: exit status: 0
[2026-03-19 19:42 INFO] MODE: Producer-Consumer with 8 consumers and 1 producers, PROCESSED FRAMES: 288
[2026-03-19 19:42 INFO] Compiling processed frames back into video...
[2026-03-19 19:42 INFO] Video successfully saved to: "processed_video.mp4"
```

Висновок. У ході виконання лабораторної роботи було практично досліджено та реалізовано ключові архітектурні патерни паралельного програмування з використанням екосистеми мови Rust. Завдяки високорівневим абстракціям фреймворку Rayon вдалося елегантно імплементувати патерни Map-Reduce, Fork-Join та Worker Pool для обчислювально інтенсивних задач, таких як аналіз тексту, розрахунок статистичних метрик та оптимізоване множення великих матриць. Використання каналів зв'язку з бібліотеки crossbeam дозволило успішно побудувати асинхронні патерни Pipeline та Producer-Consumer для конвеєрної обробки реальних відеоданих через системні виклики FFmpeg. Основним здобутком стало практичне підтвердження того, що правильний вибір паралельного патерну та його гіперпараметрів, а також дотримання принципів ізоляції стану без використання блокувань пам'яті (м'ютексів), дозволяє досягти багаторазового масштабування продуктивності програми та ефективно утилізувати всі доступні ядра сучасних процесорів.

Контрольні запитання.

1. **Поясніть різницю між м'ютексом та семафором. Коли використовувати кожен з них?**

   М'ютекс – це механізм взаємного виключення, який дозволяє лише одному потоку одночасно володіти ресурсом або виконувати критичну секцію коду. Він має суворе поняття власника: потік, який захопив м'ютекс, зобов'язаний його звільнити. Семафор (Semaphore) – це сигнальний механізм на основі лічильника, який дозволяє визначеній кількості потоків одночасно отримати доступ до пулу ресурсів. М'ютекси слід використовувати для захисту унікальних спільних даних (наприклад, єдиного словника) від одночасного запису, тоді як семафори ідеально підходять для обмеження кількості одночасних підключень до бази даних або пулу з'єднань. У мові Rust класичний Mutex інкапсулює самі дані, гарантуючи, що доступ до них можливий лише після безпосереднього блокування.

2. **Що таке потокобезпечні (thread-safe) структури даних? Наведіть приклади.**

   Потокобезпечні структури даних – це структури, внутрішня реалізація яких гарантує відсутність станів гонитви (data races) та пошкодження пам'яті при одночасному доступі з кількох паралельних потоків. Вони самостійно обробляють синхронізацію під капотом, позбавляючи розробника необхідності вручну огортати їх у зовнішні м'ютекси. У стандартній бібліотеці

Rust прикладами є канали передачі повідомлень (mpsc) та атомарні типи (AtomicUsize). В екосистемі існують високопродуктивні структури, такі як ConcurrentHashMap з крейту dashmap, яка використовує сегментоване блокування для забезпечення паралельного читання та запису без повного блокування всієї колекції, а також lock-free черги з крейту crossbeam.

3. **Поясніть різницю між песимістичною та оптимістичною синхронізацією.**

   Песимістична синхронізація виходить з припущення, що конфлікти між потоками відбуватимуться постійно. Тому потік завжди жорстко блокує ресурс перед тим, як почати з ним роботу, змушуючи всі інші потоки чекати. Це гарантує надійність, але знижує загальну продуктивність через постійні зупинки. Оптимістична синхронізація, навпаки, припускає, що конфлікти є рідкістю. Потік вільно читає дані, виконує обчислення локально, а потім намагається записати результат. Під час запису він перевіряє, чи не змінив ці дані хтось інший за час його роботи (використовуючи апаратні інструкції Compare-And-Swap). Якщо зміни відбулися, транзакція відкочується і потік повторює спробу. Це забезпечує максимальну швидкість при великій кількості читань і мінімальній кількості записів.

4. **Що таке lock-free та wait-free алгоритми?**

   Lock-free алгоритми гарантують, що щонайменше один потік у системі завжди буде прогресувати у виконанні своєї задачі, незалежно від дій або зависань інших потоків. Вони не використовують традиційних блокувань операційної системи (м'ютексів), покладаючись виключно на атомарні операції для модифікації пам'яті. Wait-free алгоритми є ще суворішою формою: вони математично гарантують, що абсолютно кожен потік завершить свою операцію за скінченну, наперед відому кількість кроків. У той час як у lock-free один потік може довго «голодувати» через те, що інші постійно виграють конкуренцію за ресурс, wait-free повністю виключає голодування. Такі алгоритми є надзвичайно складними у проектуванні та зазвичай застосовуються у системах реального часу або розробці ядер операційних систем.

5. **Поясніть патерни Producer-Consumer, Pipeline, Map-Reduce, Fork-Join, їх реалізацію.**

   Producer-Consumer відокремлює потоки, що генерують дані (виробники), від потоків, що їх обробляють (споживачі), об'єднуючи їх через спільну буферизовану чергу завдань. Pipeline розбиває складну обробку на лінійну послідовність незалежних стадій, де кожна стадія виконується у вла-

сному потоці, а дані послідовно передаються каналами. Map-Reduce розділяє роботу на фазу незалежної трансформації кожного елемента масиву даних (Map) та фазу ієрархічного злиття цих результатів в єдиний агрегований стан (Reduce). Fork-Join використовує принцип «розділяй і володарюй», рекурсивно розгалужуючи велику задачу на дрібніші паралельні підзадачі (Fork), поки вони не досягнуть базового розміру, після чого дочірні потоки завершуються, а їхні локальні результати об'єднуються на рівні вище (Join).

6. **Поясніть можливі відмінності в реалізації патернів відносно різних фреймворків.**

   Відмінності в реалізації патернів напряму залежать від парадигми конкретного середовища. Наприклад, мови з ручним управлінням потоками змушують розробника самостійно створювати пули, налаштовувати синхронізацію та стежити за станами гонитви. Натомість у Rust фреймворк Rayon абстрагує ці складнощі, автоматично реалізуючи Fork-Join через надзвичайно ефективний алгоритм work-stealing (крадіжка завдань вільними ядрами) і гарантуючи безпеку пам'яті на етапі компіляції. Крім того, у системах розподілених обчислень (наприклад, Apache Spark) патерн Map-Reduce реалізується не на рівні потоків одного процесора, а на рівні кластерів із сотень серверів. Це додає величезний шар архітектурної складності: механізми реплікації даних по мережі, серіалізацію та стійкість до падіння окремих вузлів системи, чого локальні in-memory фреймворки взагалі не потребують.

Лістинг.

html.rs

```
use crate::errors::{Error, SystemError};
use crate::task::benchmark::{BenchmarkKind, Benchmarkable};
use crate::task::executable::{Executable, RunMode};
use crate::task::lifecycle::Manageable;
use crate::task::measure::Measurable;
use crate::task::report::Reportable;
use crate::tasks::TaskLogicError;
use rand::RngExt;
use rayon::iter::IntoParallelRefIterator;
use rayon::iter::ParallelIterator;
use std::collections::HashMap;

#[derive(Debug)]
pub struct HtmlTags {
    pub documents_amount: usize,

    html_documents: Vec<String>,
}

// Builder
impl HtmlTags {
    pub fn with_documents_amount(amount: usize) -> Self {
        Self {
            documents_amount: amount,
            html_documents: vec![],
        }
    }
}

impl HtmlTags {
    // Pure function to count tags in a single document.
    // Represents the "Map" phase of our patterns.
    fn count_tags_in_doc(document: &str) -> HashMap<String, usize> {
        let mut counts = HashMap::new();

        // Very basic HTML tag extraction looking for text between '<' and '>'.
        // We split by '<', take the text up to '>', and grab the first word.
        for part in document.split('<').skip(1) {
            if let Some(tag_content) = part.split('>').next() {
                let tag_name = tag_content.split_whitespace().next().unwrap_or("");
                let tag_name = tag_name.replace("/", "").to_lowercase();

                if !tag_name.is_empty() {
                    *counts.entry(tag_name).or_insert(0) += 1;
                }
            }
        }

        counts
    }

    // Pure function to merge two HashMaps together.
    // Represents the "Reduce" or "Join" phase of our patterns.
    fn merge_counts(
    mut a: HashMap<String, usize>, b: HashMap<String, usize>,
    ) -> HashMap<String, usize> {
        for (key, value) in b {
            *a.entry(key).or_insert(0) += value;
```

```
        }
        a
    }

    // Recursive Fork-Join implementation.
    fn fork_join_recursive(docs: &[String], threshold: usize) -> HashMap<String, usize> {
        // Base case: if the chunk is small enough, process it sequentially.
        if docs.len() <= threshold {
            let mut local_counts = HashMap::new();
            for doc in docs {
                let doc_counts = Self::count_tags_in_doc(doc);
                local_counts = Self::merge_counts(local_counts, doc_counts);
            }
            return local_counts;
        }

        // Recursive case: split the workload in half.
        let mid = docs.len() / 2;
        let (left_docs, right_docs) = docs.split_at(mid);

        // Rayon's join guarantees these two closures can run on different threads.
        let (left_result, right_result) = rayon::join(
        || Self::fork_join_recursive(left_docs, threshold),
        || Self::fork_join_recursive(right_docs, threshold),
        );

        Self::merge_counts(left_result, right_result)
    }
}

impl Benchmarkable for HtmlTags {
    fn benchmarks(&self) -> Vec<BenchmarkKind> {
        vec![
        BenchmarkKind::Sequential,
        BenchmarkKind::MapReduce { partitions: 8 },
        BenchmarkKind::ForkJoin { threshold: 100 },
        BenchmarkKind::ForkJoin { threshold: 10 },
        BenchmarkKind::WorkerPool { workers: 2 },
        BenchmarkKind::WorkerPool { workers: 4 },
        BenchmarkKind::WorkerPool { workers: 8 },
        ]
    }
}

impl Manageable for HtmlTags {
    fn setup(&mut self) -> Result<(), Error> {
        let available_tags =
        ["html", "head", "body", "div", "span", "p", "a", "h1", "img"];
        let mut rng = rand::rng();

        // Generate HTML documents with random tags.
        for _ in 0..self.documents_amount {
            let mut document = String::new();
            let tags_in_document = rng.random_range(50..500);

            for _ in 0..tags_in_document {
                let tag_index = rng.random_range(0..available_tags.len());
                let tag = available_tags
                .get(tag_index)
                .ok_or(TaskLogicError::IndexOutOfBounds(tag_index))?;
                document.push_str(&format!("<{}>random text</{}> ", tag, tag));
            }

```

```
122                 self.html_documents.push(document);
123             }
124 
125             Ok(())
126         }
127 
128         fn teardown(&mut self) -> Result<(), Error> {
129             self.html_documents.clear();
130             Ok(())
131         }
132     }
133 
134     impl Executable for HtmlTags {
135         fn supported_modes(&self) -> Vec<RunMode> {
136             vec![
137             RunMode::Sequential,
138             RunMode::MapReduce,
139             RunMode::ForkJoin,
140             RunMode::WorkerPool,
141             ]
142         }
143 
144         fn run(&self, kind: BenchmarkKind) -> Result<(), Error> {
145             let unique_tags_found = match kind {
146                 BenchmarkKind::Sequential => self.sequential(),
147                 BenchmarkKind::MapReduce { .. } => self.map_reduce(),
148                 BenchmarkKind::ForkJoin { threshold } => self.fork_join(threshold),
149                 BenchmarkKind::WorkerPool { workers } => self.workers(workers),
150                 _ => return Ok(()), // Supported methods check guarantees this won't happen.
151             }?;
152 
153             log::info!(
154             "TASK: {}, MODE: {}, UNIQUE TAGS PROCESSED: {}",
155             self.name(),
156             kind,
157             unique_tags_found
158             );
159 
160             Ok(())
161         }
162     }
163 
164     // Modes
165     impl HtmlTags {
166         pub fn sequential(&self) -> Result<usize, Error> {
167             let map = self
168             .html_documents
169             .iter()
170             .map(|document| Self::count_tags_in_doc(document.as_str()))
171             .fold(HashMap::new(), Self::merge_counts);
172 
173             Ok(map.len())
174         }
175 
176         pub fn map_reduce(&self) -> Result<usize, Error> {
177             let map = self
178             .html_documents
179             .par_iter()
180             .map(|doc| Self::count_tags_in_doc(doc))
181             .reduce(HashMap::new, Self::merge_counts);
182 
183             Ok(map.len())
184         }
```

```rust

    pub fn fork_join(&self, threshold: usize) -> Result<usize, Error> {
        let map = Self::fork_join_recursive(&self.html_documents, threshold);

        Ok(map.len())
    }

    pub fn workers(&self, workers: usize) -> Result<usize, Error> {
        // Creating a custom thread pool isolated for this specific execution.
        let pool = rayon::ThreadPoolBuilder::new()
        .num_threads(workers)
        .build()
        .map_err(SystemError::WorkerPoolBuild)?;

        // Executing the parallel iteration inside our custom pool.
        let map = pool.install(|| {
            self.html_documents
            .par_iter()
            .map(|doc| Self::count_tags_in_doc(doc))
            .reduce(HashMap::new, Self::merge_counts)
        });

        Ok(map.len())
    }
}

impl Reportable for HtmlTags {
    fn name(&self) -> &'static str {
        "HTML Tag Counting"
    }
}

impl Measurable for HtmlTags {}
```

array.rs

```rust
use crate::errors::{Error, SystemError};
use crate::task::benchmark::{BenchmarkKind, Benchmarkable};
use crate::task::executable::{Executable, RunMode};
use crate::task::lifecycle::Manageable;
use crate::task::measure::Measurable;
use crate::task::report::Reportable;
use rand::RngExt;
use rayon::iter::IntoParallelRefIterator;
use rayon::iter::ParallelIterator;
use rayon::slice::ParallelSliceMut;

type Number = f64;

#[derive(Debug)]
pub struct ArrayStats {
    pub size: usize,

    numbers: Vec<Number>,
}

// Builder
impl ArrayStats {
    pub fn with_size(size: usize) -> Self {
        Self {
            size,
            numbers: vec![],
```

```rust
        }
    }
}

impl Reportable for ArrayStats {
    fn name(&self) -> &'static str {
        "Min, Max, Mean, Average of the array"
    }
}

impl Benchmarkable for ArrayStats {
    fn benchmarks(&self) -> Vec<BenchmarkKind> {
        vec![
        BenchmarkKind::Sequential,
        BenchmarkKind::MapReduce { partitions: 8 },
        BenchmarkKind::ForkJoin { threshold: 10_000 },
        BenchmarkKind::ForkJoin { threshold: 1_000 },
        BenchmarkKind::WorkerPool { workers: 2 },
        BenchmarkKind::WorkerPool { workers: 4 },
        BenchmarkKind::WorkerPool { workers: 8 },
        ]
    }
}

impl Manageable for ArrayStats {
    fn setup(&mut self) -> Result<(), Error> {
        let mut rng = rand::rng();

        // Generate a massive array of random floats.
        // We reserve capacity to avoid reallocation overhead.
        self.numbers.reserve_exact(self.size);
        for _ in 0..self.size {
            // Generating values between -1000.0 and 1000.0
            self.numbers.push(rng.random_range(-1000.0..1000.0));
        }

        Ok(())
    }

    fn teardown(&mut self) -> Result<(), Error> {
        self.numbers.clear();
        self.numbers.shrink_to_fit();
        Ok(())
    }
}

impl Executable for ArrayStats {
    fn supported_modes(&self) -> Vec<RunMode> {
        vec![
        RunMode::Sequential,
        RunMode::MapReduce,
        RunMode::ForkJoin,
        RunMode::WorkerPool,
        ]
    }

    fn run(&self, kind: BenchmarkKind) -> Result<(), Error> {
        let stats = match kind {
            BenchmarkKind::Sequential => self.sequential(),
            BenchmarkKind::MapReduce { .. } => self.map_reduce(),
            BenchmarkKind::ForkJoin { threshold } => self.fork_join(threshold),
            BenchmarkKind::WorkerPool { workers } => self.workers(workers),
            _ => return Ok(()),
```

```rust
        }?;

        log::info!(
        "TASK: {}, MODE: {}, STATS: Min: {:.2}, Max: {:.2}, Mean: {:.2}, Median: {:.2}",
        self.name(),
        kind,
        stats.min,
        stats.max,
        stats.mean,
        stats.median
        );

        Ok(())
    }
}

impl Measurable for ArrayStats {}

// A helper struct to accumulate our metrics during parallel execution.
#[derive(Debug, Clone, Copy)]
pub struct StatsAccumulator {
    min: Number,
    max: Number,
    sum: Number,
    count: usize,
}

impl StatsAccumulator {
    // Initializes the accumulator with a single value.
    fn from_value(value: Number) -> Self {
        Self {
            min: value,
            max: value,
            sum: value,
            count: 1,
        }
    }

    // Merges two accumulators together, which represents our "Reduce" phase.
    fn merge(self, other: Self) -> Self {
        Self {
            min: self.min.min(other.min),
            max: self.max.max(other.max),
            sum: self.sum + other.sum,
            count: self.count + other.count,
        }
    }
}

// A struct to format the final result for our logs.
#[derive(Debug)]
pub struct FinalStats {
    min: Number,
    max: Number,
    mean: Number,
    median: Number,
}

// Helpers
impl ArrayStats {
    // Recursive Fork-Join implementation for finding min, max, and sum.
    fn fork_join_recursive(data: &[f64], threshold: usize) -> StatsAccumulator {
        // Base case: process sequentially if the chunk is small enough.
```

```rust
        if data.len() <= threshold {
            let mut acc = StatsAccumulator::from_value(data[0]);
            for &val in data.iter().skip(1) {
                acc = acc.merge(StatsAccumulator::from_value(val));
            }
            return acc;
        }

        // Recursive case: split the workload in half.
        let mid = data.len() / 2;
        let (left_data, right_data) = data.split_at(mid);

        // Rayon's join executes closures on potentially different threads.
        let (left_result, right_result) = rayon::join(
        || Self::fork_join_recursive(left_data, threshold),
        || Self::fork_join_recursive(right_data, threshold),
        );

        left_result.merge(right_result)
    }

    // Helper function to calculate the median after sorting.
    fn calculate_median(sorted_data: &[f64]) -> f64 {
        let len = sorted_data.len();
        if len.is_multiple_of(2) {
            (sorted_data[len / 2 - 1] + sorted_data[len / 2]) / 2.0
        } else {
            sorted_data[len / 2]
        }
    }
}

// Modes
impl ArrayStats {
    pub fn sequential(&self) -> Result<FinalStats, Error> {
        // Find min, max, and sum using standard sequential folding.
        let initial = StatsAccumulator::from_value(self.numbers[0]);
        let accumulator = self
        .numbers
        .iter()
        // Because we already initialized with it
        .skip(1)
        .fold(initial, |acc, &val| acc.merge(StatsAccumulator::from_value(val)));

        // Clone and sort sequentially for the median.
        let mut local_copy = self.numbers.clone();
        local_copy.sort_unstable_by(|a, b| a.total_cmp(b));
        let median = Self::calculate_median(&local_copy);

        Ok(FinalStats {
            min: accumulator.min,
            max: accumulator.max,
            mean: accumulator.sum / accumulator.count as f64,
            median,
        })
    }

    pub fn map_reduce(&self) -> Result<FinalStats, Error> {
        // Map-Reduce logic using Rayon's parallel iterators.
        let acc = self
        .numbers
        .par_iter()
        .map(|&val| StatsAccumulator::from_value(val))
```

```rust
        .reduce(
        || StatsAccumulator::from_value(self.numbers[0]),
        |a, b| a.merge(b),
        );

        // Clone and sort in parallel using Rayon.
        let mut local_copy = self.numbers.clone();
        local_copy.par_sort_unstable_by(|a, b| a.total_cmp(b));
        let median = Self::calculate_median(&local_copy);

        Ok(FinalStats {
            min: acc.min,
            max: acc.max,
            mean: acc.sum / acc.count as f64,
            median,
        })
    }

    pub fn fork_join(&self, threshold: usize) -> Result<FinalStats, Error> {
        let acc = Self::fork_join_recursive(&self.numbers, threshold);

        // Utilizing parallel sort for the median to maintain performance.
        let mut local_copy = self.numbers.clone();
        local_copy.par_sort_unstable_by(|a, b| a.total_cmp(b));
        let median = Self::calculate_median(&local_copy);

        Ok(FinalStats {
            min: acc.min,
            max: acc.max,
            mean: acc.sum / acc.count as f64,
            median,
        })
    }

    pub fn workers(&self, workers: usize) -> Result<FinalStats, Error> {
        let pool = rayon::ThreadPoolBuilder::new()
        .num_threads(workers)
        .build()
        .map_err(SystemError::WorkerPoolBuild)?;

        let final_stats = pool.install(|| {
            let acc = self
            .numbers
            .par_iter()
            .map(|&val| StatsAccumulator::from_value(val))
            .reduce(
            || StatsAccumulator::from_value(self.numbers[0]),
            |a, b| a.merge(b),
            );

            let mut local_copy = self.numbers.clone();
            local_copy.par_sort_unstable_by(|a, b| a.total_cmp(b));
            let median = Self::calculate_median(&local_copy);

            FinalStats {
                min: acc.min,
                max: acc.max,
                mean: acc.sum / acc.count as f64,
                median,
            }
        });

        Ok(final_stats)
```

```rust
    }
}
```

multiplication.rs

```rust
use crate::errors::{Error, SystemError};
use crate::task::benchmark::{BenchmarkKind, Benchmarkable};
use crate::task::executable::{Executable, RunMode};
use crate::task::lifecycle::Manageable;
use crate::task::measure::Measurable;
use crate::task::report::Reportable;
use rand::RngExt;
use rayon::iter::{IntoParallelIterator, ParallelIterator};

#[derive(Debug)]
pub struct MatrixMultiplication {
    pub size: usize,

    matrix_a: Vec<f64>,
    matrix_b: Vec<f64>,
}

// Builder
impl MatrixMultiplication {
    pub fn with_size(size: usize) -> Self {
        Self {
            size,

            matrix_a: vec![],
            matrix_b: vec![],
        }
    }
}

impl MatrixMultiplication {
    // Pure function to multiply a single row of matrix A with all rows of transposed matrix B.
    // Represents the "Map" phase of our execution.
    fn compute_row(size: usize, row_a: &[f64], matrix_b_transposed: &[f64]) -> Vec<f64> {
        let mut result_row = Vec::with_capacity(size);

        for j in 0..size {
            let mut sum = 0.0;
            // Both row_a and matrix_b_transposed are accessed sequentially here,
            // resulting in perfect CPU cache utilization.
            let row_b = &matrix_b_transposed[j * size..(j + 1) * size];
            for k in 0..size {
                sum += row_a[k] * row_b[k];
            }
            result_row.push(sum);
        }

        result_row
    }

    // Pure function to merge two chunks of matrix rows together.
    // Represents the "Reduce" or "Join" phase.
    fn merge_matrix_chunks(mut chunk_a: Vec<f64>, chunk_b: Vec<f64>) -> Vec<f64> {
        chunk_a.extend(chunk_b);
        chunk_a
    }

    // Transposes the matrix to optimize memory access patterns during multiplication.
```

```rust
    fn transpose(size: usize, matrix: &[f64]) -> Vec<f64> {
        let mut transposed = vec![0.0; size * size];
        for i in 0..size {
            for j in 0..size {
                transposed[j * size + i] = matrix[i * size + j];
            }
        }
        transposed
    }

    // Recursive Fork-Join implementation.
    // We recursively split the workload by dividing the rows of the resulting matrix.
    fn fork_join_recursive(
    size: usize, matrix_a: &[f64], matrix_b_t: &[f64], row_start: usize,
    row_end: usize, threshold: usize,
    ) -> Vec<f64> {
        let row_count = row_end - row_start;

        // Base case: process sequentially if the chunk of rows is small enough.
        if row_count <= threshold {
            let mut result_chunk = Vec::with_capacity(row_count * size);
            for i in row_start..row_end {
                let row_a = &matrix_a[i * size..(i + 1) * size];
                let computed_row = Self::compute_row(size, row_a, matrix_b_t);
                result_chunk.extend(computed_row);
            }
            return result_chunk;
        }

        // Recursive case: split the row range in half.
        let mid = row_start + row_count / 2;

        // Rayon's join executes closures on potentially different threads.
        let (left_result, right_result) = rayon::join(
        || {
            Self::fork_join_recursive(
            size, matrix_a, matrix_b_t, row_start, mid, threshold,
            )
        },
        || {
            Self::fork_join_recursive(
            size, matrix_a, matrix_b_t, mid, row_end, threshold,
            )
        },
        );

        Self::merge_matrix_chunks(left_result, right_result)
    }
}

impl Benchmarkable for MatrixMultiplication {
    fn benchmarks(&self) -> Vec<BenchmarkKind> {
        vec![
        BenchmarkKind::Sequential,
        BenchmarkKind::MapReduce { partitions: 8 },
        BenchmarkKind::ForkJoin { threshold: 100 },
        BenchmarkKind::ForkJoin { threshold: 10 },
        BenchmarkKind::WorkerPool { workers: 2 },
        BenchmarkKind::WorkerPool { workers: 4 },
        BenchmarkKind::WorkerPool { workers: 8 },
        ]
    }
}
```

```rust

impl Manageable for MatrixMultiplication {
    fn setup(&mut self) -> Result<(), Error> {
        let mut rng = rand::rng();
        let total_elements = self.size * self.size;

        self.matrix_a.reserve_exact(total_elements);
        self.matrix_b.reserve_exact(total_elements);

        for _ in 0..total_elements {
            self.matrix_a.push(rng.random_range(-10.0..10.0));
            self.matrix_b.push(rng.random_range(-10.0..10.0));
        }

        Ok(())
    }

    fn teardown(&mut self) -> Result<(), Error> {
        self.matrix_a.clear();
        self.matrix_a.shrink_to_fit();
        self.matrix_b.clear();
        self.matrix_b.shrink_to_fit();
        Ok(())
    }
}

impl Executable for MatrixMultiplication {
    fn supported_modes(&self) -> Vec<RunMode> {
        vec![
        RunMode::Sequential,
        RunMode::MapReduce,
        RunMode::ForkJoin,
        RunMode::WorkerPool,
        ]
    }

    fn run(&self, kind: BenchmarkKind) -> Result<(), Error> {
        let result_matrix = match kind {
            BenchmarkKind::Sequential => self.sequential(),
            BenchmarkKind::MapReduce { .. } => self.map_reduce(),
            BenchmarkKind::ForkJoin { threshold } => self.fork_join(threshold),
            BenchmarkKind::WorkerPool { workers } => self.workers(workers),
            _ => return Ok(()),
        }?;

        // A quick validation to ensure the matrix is actually computed.
        // We log the value of the very first element.
        let first_element = result_matrix.first().copied().unwrap_or(0.0);

        log::info!(
        "TASK: {}, MODE: {}, FIRST ELEMENT VALUE: {:.2}",
        self.name(),
        kind,
        first_element
        );

        Ok(())
    }
}

// Modes
impl MatrixMultiplication {
    pub fn sequential(&self) -> Result<Vec<f64>, Error> {
```

```
        let matrix_b_t = Self::transpose(self.size, &self.matrix_b);

        // Fold here accumulates rows into the final flat vector.
        let result = (0..self.size)
        .map(|i| {
            let row_a = &self.matrix_a[i * self.size..(i + 1) * self.size];
            Self::compute_row(self.size, row_a, &matrix_b_t)
        })
        .fold(
        Vec::with_capacity(self.size * self.size),
        Self::merge_matrix_chunks,
        );

        Ok(result)
    }

    pub fn map_reduce(&self) -> Result<Vec<f64>, Error> {
        let matrix_b_t = Self::transpose(self.size, &self.matrix_b);

        // Parallel iteration over row indices.
        let result = (0..self.size)
        .into_par_iter()
        .map(|i| {
            let row_a = &self.matrix_a[i * self.size..(i + 1) * self.size];
            Self::compute_row(self.size, row_a, &matrix_b_t)
        })
        .reduce(Vec::new, Self::merge_matrix_chunks);

        Ok(result)
    }

    pub fn fork_join(&self, threshold: usize) -> Result<Vec<f64>, Error> {
        let matrix_b_t = Self::transpose(self.size, &self.matrix_b);
        let result = Self::fork_join_recursive(
        self.size,
        &self.matrix_a,
        &matrix_b_t,
        0,
        self.size,
        threshold,
        );

        Ok(result)
    }

    pub fn workers(&self, workers: usize) -> Result<Vec<f64>, Error> {
        let matrix_b_t = Self::transpose(self.size, &self.matrix_b);

        let pool = rayon::ThreadPoolBuilder::new()
        .num_threads(workers)
        .build()
        .map_err(|_| SystemError::CurrentExe(std::io::Error::last_os_error()))?;

        let result = pool.install(|| {
            (0..self.size)
            .into_par_iter()
            .map(|i| {
                let row_a = &self.matrix_a[i * self.size..(i + 1) * self.size];
                Self::compute_row(self.size, row_a, &matrix_b_t)
            })
            .reduce(Vec::new, Self::merge_matrix_chunks)
        });

```

```rust
        Ok(result)
    }
}

impl Reportable for MatrixMultiplication {
    fn name(&self) -> &'static str {
        "Matrices Multiplication"
    }
}

impl Measurable for MatrixMultiplication {}
```

video.rs

```rust
use crate::errors::{Error, SystemError};
use crate::task::benchmark::{BenchmarkKind, Benchmarkable};
use crate::task::executable::{Executable, RunMode};
use crate::task::lifecycle::Manageable;
use crate::task::measure::Measurable;
use crate::task::report::Reportable;
use image::{DynamicImage, RgbaImage, imageops};
use std::fs;
use std::path::{Path, PathBuf};
use std::process::Command;
use std::thread;

#[derive(Debug)]
pub struct VideoPipeline {
    input_video: PathBuf,
    input_directory: PathBuf,
    output_directory: PathBuf,
    watermark: RgbaImage,
    frame_paths: Vec<PathBuf>,
}

impl VideoPipeline {
    const BASE_DIRECTORY: &'static str = "kpi_video_lab";
    const FRAMES_INPUT_DIRECTORY: &'static str = "frames_in";
    const FRAMES_OUTPUT_DIRECTORY: &'static str = "frames_out";

    // You should pass the path to a short test video (e.g., 5-10 seconds)
    pub fn with_video(video_path: &str) -> Self {
        let base_dir = std::env::temp_dir().join(Self::BASE_DIRECTORY);

        Self {
            input_video: PathBuf::from(video_path),
            input_directory: base_dir.join(Self::FRAMES_INPUT_DIRECTORY),
            output_directory: base_dir.join(Self::FRAMES_OUTPUT_DIRECTORY),
            // Generating a simple semi-transparent red 100x100 watermark
            watermark: RgbaImage::from_pixel(100, 100, image::Rgba([255, 0, 0, 128])),
            frame_paths: vec![],
        }
    }
}

#[derive(Debug)]
pub struct DynamicImageResource {
    pub image: DynamicImage,
    pub path: PathBuf,
}

#[derive(Debug)]
```

```rust
pub struct RgbaImageResource {
    pub image: RgbaImage,
    pub path: PathBuf,
}

// Pure functions representing our processing stages
impl VideoPipeline {
    // Stage 1: Decode (Read image file from disk into memory)
    fn decode_stage(path: &Path) -> Result<DynamicImageResource, Error> {
        let image = image::open(path).map_err(SystemError::ImageOpen)?;

        Ok(DynamicImageResource {
            image,
            path: path.to_path_buf(),
        })
    }

    // Stage 2: Apply a grayscale filter
    fn filter_stage(resource: DynamicImageResource) -> DynamicImageResource {
        DynamicImageResource {
            image: resource.image.grayscale(),
            path: resource.path,
        }
    }

    // Stage 3: Apply the watermark
    fn watermark_stage(
    resource: DynamicImageResource, watermark: &RgbaImage,
    ) -> RgbaImageResource {
        let mut rgba_image = resource.image.to_rgba8();

        // Overlay watermark at top-left corner
        imageops::overlay(&mut rgba_image, watermark, 10, 10);

        RgbaImageResource {
            image: rgba_image,
            path: resource.path,
        }
    }

    // Stage 4: Encode (Save the processed image back to disk)
    fn encode_stage(
    data: RgbaImageResource, output_directory: &Path,
    ) -> Result<(), Error> {
        let file_name = data.path.file_name().ok_or(SystemError::FileName(
        data.path.to_string_lossy().to_string(),
        ))?;

        let image_path = output_directory.join(file_name);

        data.image
        .save(image_path)
        .map_err(SystemError::ImageSave)
        .map_err(Error::System)
    }
}

impl Benchmarkable for VideoPipeline {
    fn benchmarks(&self) -> Vec<BenchmarkKind> {
        vec![
        BenchmarkKind::Sequential,
        BenchmarkKind::Pipeline {
            buffer_capacity: 5,
```

```
112                 max_threads: 4,
113             },
114             BenchmarkKind::ProducerConsumer {
115                 buffer_capacity: 20,
116                 producers: 1,
117                 consumers: 2,
118             },
119             BenchmarkKind::ProducerConsumer {
120                 buffer_capacity: 20,
121                 producers: 1,
122                 consumers: 4,
123             },
124             BenchmarkKind::ProducerConsumer {
125                 buffer_capacity: 20,
126                 producers: 1,
127                 consumers: 8,
128             },
129             ]
130         }
131     }
132 
133     impl Manageable for VideoPipeline {
134         fn setup(&mut self) -> Result<(), Error> {
135             fs::create_dir_all(&self.input_directory)
136             .map_err(SystemError::DirectoryCreate)?;
137             fs::create_dir_all(&self.output_directory)
138             .map_err(SystemError::DirectoryCreate)?;
139 
140             // Extract frames ONLY if the directory is empty to save time across benchmarks
141             let is_empty = fs::read_dir(&self.input_directory)
142             .map_err(SystemError::DirectoryRead)?
143             .next()
144             .is_none();
145 
146             if is_empty {
147                 log::info!("Extracting frames from video using FFmpeg...");
148                 let status = Command::new("ffmpeg")
149                 .arg("-i")
150                 .arg(&self.input_video)
151                 // Hardware scale filter to resize frames to 480p width.
152                 .arg("-vf")
153                 .arg("scale=854:-1")
154                 // Outputting as PNG. 480p PNGs will easily fit in the tmpfs RAM disk.
155                 .arg(self.input_directory.join("frame_%04d.png"))
156                 .status()
157                 .map_err(SystemError::FfmpegExecution)?;
158                 log::info!("FFmpeg exited with status: {}", status);
159             }
160 
161             self.frame_paths.clear();
162 
163             for entry in fs::read_dir(&self.input_directory)
164             .map_err(SystemError::DirectoryRead)?
165             .flatten()
166             {
167                 self.frame_paths.push(entry.path());
168             }
169 
170             // Sorting guarantees the frames are processed sequentially
171             self.frame_paths.sort();
172 
173             Ok(())
174         }
```

```rust

    fn teardown(&mut self) -> Result<(), Error> {
        log::info!("Compiling processed frames back into video...");

        // Output video path (saving in the current working directory).
        // Feel free to change this path if you want it saved elsewhere.
        const FINAL_VIDEO_NAME: &str = "processed_video.mp4";
        let final_video_path = PathBuf::from(FINAL_VIDEO_NAME);

        // Execute FFmpeg to compile the frames.
        // -y: Overwrite output file if it exists.
        // -r 24: Set framerate to 24 fps (adjust if your source video is different).
        // -i ...: Input pattern for the frames.
        // -c:v libx264: Use standard H.264 codec for max compatibility.
        // -pix_fmt yuv420p: Ensure pixel format is compatible with all players.
        let status = Command::new("ffmpeg")
        .arg("-y")
        .arg("-r")
        .arg("24")
        .arg("-i")
        .arg(self.output_directory.join("frame_%04d.png"))
        .arg("-c:v")
        .arg("libx264")
        .arg("-vf")
        .arg("pad=ceil(iw/2)*2:ceil(ih/2)*2")
        .arg("-pix_fmt")
        .arg("yuv420p")
        .arg(&final_video_path)
        .status()
        .map_err(SystemError::FfmpegExecution)?;

        if status.success() {
            log::info!("Video successfully saved to: {:?}", final_video_path);
        } else {
            log::error!("FFmpeg compilation failed with status: {}", status);
        }

        // Clean up temporary directories.
        fs::remove_dir_all(&self.input_directory)
        .map_err(SystemError::DirectoryRemove)
        .map_err(Error::System)?;

        fs::remove_dir_all(&self.output_directory)
        .map_err(SystemError::DirectoryRemove)
        .map_err(Error::System)
    }
}

impl Executable for VideoPipeline {
    fn supported_modes(&self) -> Vec<RunMode> {
        vec![
        RunMode::Sequential,
        RunMode::Pipeline,
        RunMode::ProducerConsumer,
        ]
    }

    fn run(&self, kind: BenchmarkKind) -> Result<(), Error> {
        let count = match kind {
            BenchmarkKind::Sequential => self.sequential(),
            BenchmarkKind::Pipeline { .. } => self.pipeline(),
            BenchmarkKind::ProducerConsumer { consumers, .. } => {
                self.producer_consumer(consumers)
```

```
            },
            _ => return Ok(()),
        }?;

        log::info!(
        "TASK: {}, MODE: {}, PROCESSED FRAMES: {}",
        self.name(),
        kind,
        count
        );
        Ok(())
    }
}

// Execution Modes
impl VideoPipeline {
    pub fn sequential(&self) -> Result<usize, Error> {
        let mut count = 0;
        for path in &self.frame_paths {
            let data = Self::decode_stage(path)?;
            let data = Self::filter_stage(data);
            let data = Self::watermark_stage(data, &self.watermark);
            Self::encode_stage(data, &self.output_directory)?;
            count += 1;
        }
        Ok(count)
    }

    pub fn pipeline(&self) -> Result<usize, Error> {
        // Crossbeam bounded channels for strict memory control between stages
        let cap = 5;
        let (raw_tx, raw_rx) = crossbeam::channel::bounded(cap);
        let (decoded_tx, decoded_rx) = crossbeam::channel::bounded(cap);
        let (filtered_tx, filtered_rx) = crossbeam::channel::bounded(cap);

        let (errors_tx, errors_rx) = crossbeam::channel::bounded(1);

        thread::scope(|s| {
            // Stage 1: Decode
            s.spawn(move || {
                for path in self.frame_paths.iter() {
                    let decoded = Self::decode_stage(path);

                    if raw_tx.send(decoded).is_err() {
                        break;
                    }
                }
            });

            // Stage 2: Filter
            s.spawn({
                let errors_tx = errors_tx.clone();
                move || {
                    for data in raw_rx {
                        let data = match data {
                            Ok(data) => data,
                            Err(error) => {
                                let _ = errors_tx.try_send(error);
                                break;
                            },
                        };

                        let filtered = Self::filter_stage(data);
```

```
                        if decoded_tx.send(filtered).is_err() {
                            break;
                        }
                    }
                }
            });

            // Stage 3: Watermark
            s.spawn(move || {
                for data in decoded_rx {
                    let watermarked = Self::watermark_stage(data, &self.watermark);
                    if filtered_tx.send(watermarked).is_err() {
                        break;
                    }
                }
            });

            // Stage 4: Encode
            s.spawn({
                let errors_tx = errors_tx.clone();
                move || {
                    let mut count = 0;
                    for data in filtered_rx {
                        match Self::encode_stage(data, &self.output_directory) {
                            Ok(_) => count += 1,
                            Err(error) => {
                                let _ = errors_tx.try_send(error);
                                break;
                            },
                        }
                    }
                    count
                }
            });
        });

        if let Ok(error) = errors_rx.try_recv() {
            return Err(error);
        }

        Ok(self.frame_paths.len())
    }

    pub fn producer_consumer(&self, consumers: usize) -> Result<usize, Error> {
        let cap = 20;
        let (tasks_tx, tasks_rx) = crossbeam::channel::bounded(cap);
        let (results_tx, results_rx) =
        crossbeam::channel::bounded(self.frame_paths.len());
        let (errors_tx, errors_rx) = crossbeam::channel::bounded(1);

        thread::scope(|s| {
            // Producer
            s.spawn(move || {
                for path in self.frame_paths.iter() {
                    if tasks_tx.send(path).is_err() {
                        break;
                    }
                }
            });

            // Consumers
            for _ in 0..consumers {
                let tasks_rx = tasks_rx.clone();
```

```rust
                let results_tx = results_tx.clone();
                let errors_tx = errors_tx.clone();

                let watermark = self.watermark.clone();
                let output_directory = self.output_directory.clone();

                s.spawn(move || {
                    for path in tasks_rx {
                        let data = match Self::decode_stage(path) {
                            Ok(data) => data,
                            Err(error) => {
                                let _ = errors_tx.try_send(error);
                                break;
                            },
                        };
                        let data = Self::filter_stage(data);
                        let data = Self::watermark_stage(data, &watermark);
                        match Self::encode_stage(data, &output_directory) {
                            Ok(_) => {
                                if results_tx.send(1).is_err() {
                                    break;
                                }
                            },
                            Err(error) => {
                                let _ = errors_tx.try_send(error);
                                break;
                            },
                        }
                    }
                });
            }
        });

        if let Ok(error) = errors_rx.try_recv() {
            return Err(error);
        }

        // CRITICAL FIX: Explicitly drop the original channel ends held by the main thread.
        // This ensures the receivers know no more messages will ever be sent, preventing infinite blocking.
        drop(tasks_rx);
        drop(results_tx);
        drop(errors_tx);

        let mut total = 0;
        for _ in results_rx {
            total += 1;
        }

        Ok(total)
    }
}

impl Reportable for VideoPipeline {
    fn name(&self) -> &'static str {
        "Video Processing Pipeline"
    }
}

impl Measurable for VideoPipeline {}
```